

Dokumentation

Ha-Py Virtuell

1. Die Systemarchitektur.....	2
1.1 Das Unity-Projekt.....	3
1.2 Die App Stores.....	3
1.3 Die App.....	3
1.4 Die JSON-Schnittstelle.....	3
1.5 Die Medienordner.....	5
1.6 Das Redaktionssystem.....	5
2. Code-Dokumentation zum Unity-Projekt.....	6
2.1 Frontend-Umsetzung.....	6
2.2 Standardfunktionen.....	6
2.3 Medienfunktionen.....	7
2.4 AR-Funktionen.....	7
3. Datenstruktur-Dokumentation.....	10
3.1 Datenstruktur der Unity-App.....	10
3.2 Datenstruktur des CMS.....	10
4. Versionsverwaltung.....	12
4.1 Versionierung der Unity-App.....	12
4.2 Versionierung des CMS.....	12
5. Übergabe der Projektergebnisse.....	13
5.1 Übergabe des Unity-Projekts.....	13
5.2 Übergabe der JSONs.....	13
5.3 Übergabe des Redaktionssystems (Schulung).....	14
5.3.1 Fokus auf "Inhalte".....	14
5.3.2 Neue Touren anlegen.....	15
5.3.3 Neue Stationen anlegen b.z.w. verknüpfen.....	16
5.3.4 Neue Medieninhalte anlegen b.z.w. verknüpfen.....	16
5.3.5 Vorgaben für die Medieninhalte.....	17

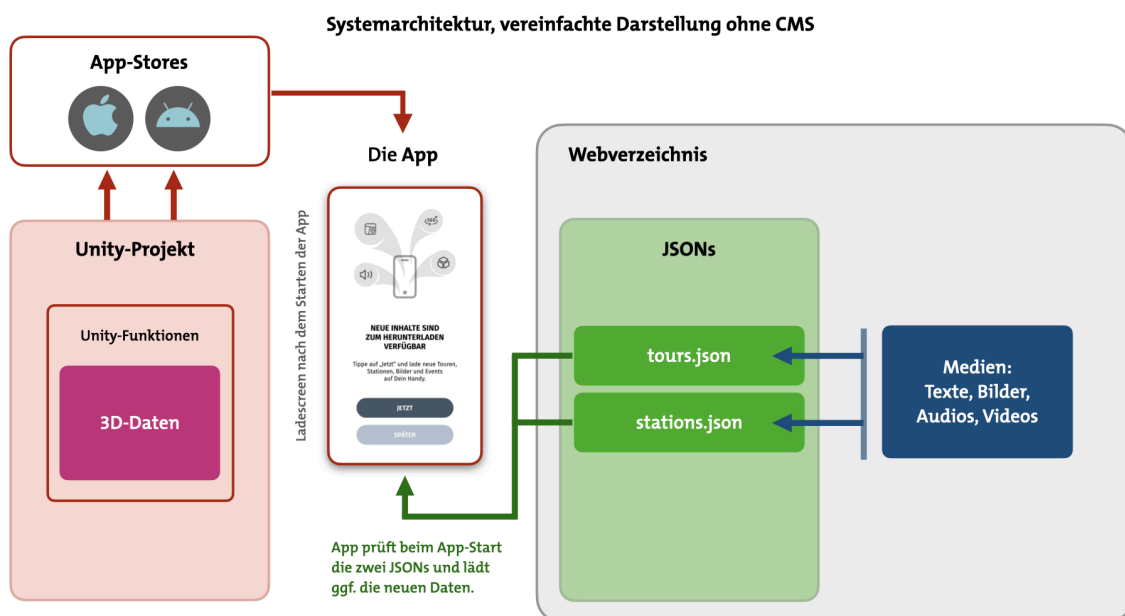
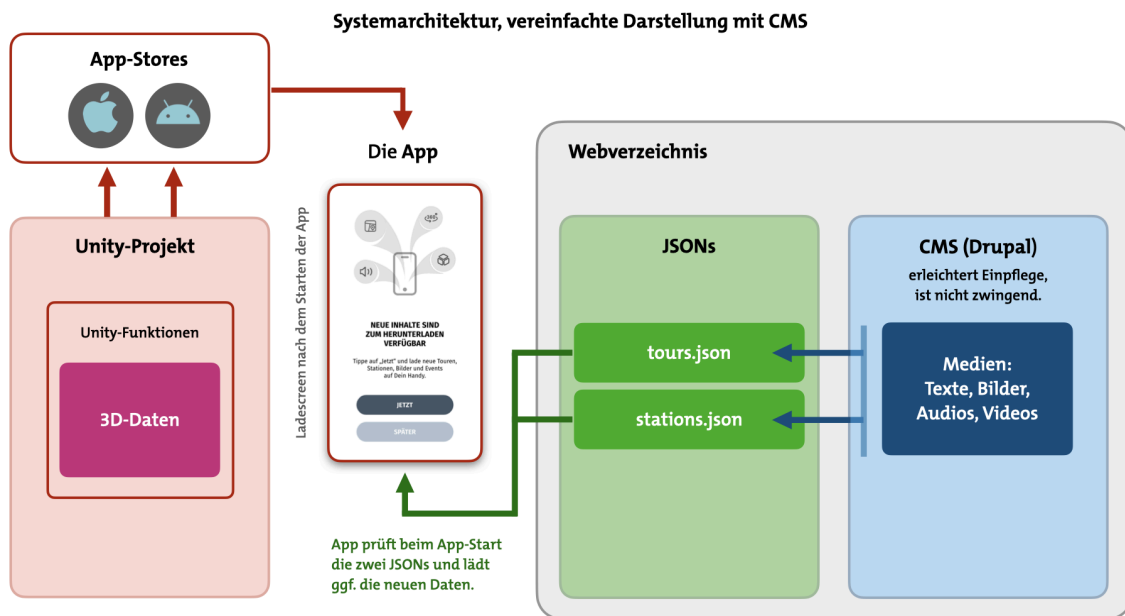
Vorbemerkung

Die Kernaufgabe der App "Ha-Py Virtuell" ist die unterhaltsame Vermittlung von Inhalten mittels Augmented Reality (AR). In der initial umgesetzten Tour wird ein virtueller Rauchgaskegel dargestellt und kann virtuell "begangen" werden. Außerdem tritt in der Station "A" ein virtueller AR-Avatar als Moderator auf.

Ergänzend zu den AR-Darstellungen wurden "klassische" Medieninhalte integriert: Texte, Bilder, Audiodateien und 360-Grad-Bilder. Und schließlich gibt es außerdem zwei mit variablen Inhalten bestückbare Spielefunktionen (Bilderrätsel und Fragequiz), die in beliebiger Anzahl an den Stationen integriert werden können.

1. Die Systemarchitektur

Die Systemarchitektur besteht aus einem **Unity-Projekt**, mit dem die **App** generiert wird, den **App-Stores** als Veröffentlichungsinstanz und einem **Webverzeichnis** mit einer **JSON-Schnittstelle** und der **Medienablage**, die standardmäßig durch ein Redaktionssystem (**CMS**) verwaltet wird. Letzteres gehört aber nicht zum Kern der Systemarchitektur, da die App und ihre Aktualisierung über die JSON-Schnittstelle auch problemlos ohne die Komfortlösungen eines Redaktionssystems funktionieren kann.

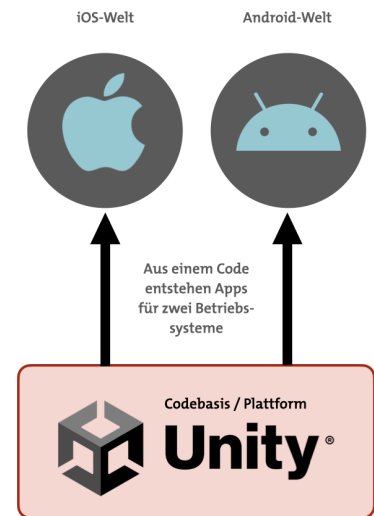


1.1 Das Unity-Projekt

Die App wird in Unity programmiert, eine leistungsstarke, plattformübergreifende Entwicklungsumgebung, die für die Erstellung von Spielen und interaktiven Anwendungen genutzt wird. Sie ermöglicht, basierend auf einer zentralen Codes-Basis, die Ausspielung der App für verschiedene Plattformen, in Fall der Ha-Py-App für iOS und Android.

Mit Unity können die Entwickler realistische Grafiken, Animationen und physikalische Effekte umsetzen. Durch die integrierte Unterstützung von Augmented Reality (AR) und Virtual Reality (VR) ist Unity eine bevorzugte Wahl für AR-Projekte, da es nahtlos immersive Erlebnisse auf verschiedenen Geräten bereitstellt.

Details zur Unity-Programmierung werden im Kapitel “Code-Dokumentation” erläutert. Die detailliertere Darstellung der Systemarchitektur zeigt die Quellenlage und Verknüpfung mit weiteren Daten, die in den folgenden Absätzen näher erläutert werden.



1.2 Die App Stores

Die App ist für die Betriebssysteme iOS und Android in den entsprechenden App Stores verfügbar. Als Herausgeber und Verantwortlicher für die App wurde der Landkreis Hameln-Pyrmont eingetragen. Die Expo-Etage unter dem Dach der Mobile-Etage GmbH wurde zur Veröffentlichung der App den Entwickler-Accounts des Landkreises b.z.w. des Ha-Py-Projekts hinzugefügt. Etwaige Korrespondenzen mit den Betreibern der App Stores und die Pflege der App obliegt dem Landkreis Hameln-Pyrmont als Auftraggeber. Als Dienstleister können wir im Bedarfsfall und gegen Aufwandsentschädigung gerne unterstützen.

1.3 Die App

Die App ist für moderne Endgeräte nutzbar, die eine **AR-Funktionalität** entweder vorinstalliert haben (wie iOS-Smartphones) oder diese durch die Installation von AR-Core (Android) ergänzt haben. Die Darstellung von live gerenderten 3D-Modellen benötigt eine **passable Performance** der Endgeräte.

Das **3D-Modell des Rauchgaskegels** ist im Unity-Projekt integriert und wird bei der Installation der App zusammen mit allen sonstigen App-Bestandteilen heruntergeladen. Die Programmierung der App wird in dem Abschnitt “Code-Dokumentation” näher beschrieben.

1.4 Die JSON-Schnittstelle

Die App erhält ihre Inhalte über zwei **JSON-Dateien**: “**tours.json**” und “**stations.json**”. Sie greift auf diese JSONs einmal initial nach der Installation der App auf dem Endgerät zu und später dann optional nach dem Starten der App. Durch diese smarte Aktualisierungsfunktion können inhaltliche Änderungen an der App ohne Update in den App-Stores erfolgen.

Die beiden JSON-Dateien enthalten alle App-Texte und Verlinkungen auf Medien. Die App greift auf die JSON-Dateien über einfache **Web-Requests** zu. Dazu wurde ein **ServerBackendController**-Script geschrieben. Über fest definierte Server-URLs und einer Passwort-Autorisierung werden die JSON-Dateien beim Start der App heruntergeladen. Alle Weblinks auf Medien werden herausgefiltert und können dann nach Bestätigung vom Nutzer zusätzlich heruntergeladen werden, sodass sie direkt innerhalb der Stationen abrufbar sind.

- Diese beiden Dateien enthalten den textbasierten Content der App. Neben den **Inhaltstexten** auf den Seiten der App sind das auch die **Texte des User Interfaces**, zum Beispiel Buttontexte, Tutorialtexte und Hilfetexte.
- Auch die **Gliederung der App** stellt sich vollumfänglich und gut strukturiert in der JSON dar.
 - Die **JSON “tours.json”** listet die integrierten Touren und ihre Detailinformationen auf. Dazu gehören beispielsweise Titel, Länge in km, Dauer in Stunden und pro Tour ein Beschreibungstext. Auch werden in der “tours.js” die **Stationen** zugeordnet und verknüpft.
 - Die **JSON “stations.json”** listet unter anderem alle Basisinformationen zu den Stationen auf. Dazu gehören beispielsweise Titel, Kurzbeschreibung und GPS-Koordinaten. Aber auch alle **Features** innerhalb der Stationen werden genannt, um im App-Frontend im 3-Punkt-Menü zu erscheinen. Teilweise stehen die Informationen zu den Features direkt als **Klartext** in der JSON, teilweise ruft die JSON aber auch nur **Funktionen** auf, die im Programmcode der App weiter verhandelt werden. Letzteres betrifft zum Beispiel die AR-Module, die Teil des von Unity generierten Programmcodes sind.
- Beide JSONs enthalten außerdem Verweise auf die **Mediendateien** (Bilder, Audios und Videos), die unabhängig von der JSON im Webverzeichnis des Projekts liegen, um von dort aus initial in die App geladen zu werden.

1.5 Die Medienordner

Die klassischen Medieninhalte der App (Bilder, Audios, Videos) liegen im Webverzeichnis des Projekts und werden mittels der JSONs auf die Endgeräte heruntergeladen. Dadurch ist die App während des Gebrauchs unabhängig von einer Internetverbindung.

Die Darstellung der Medien in der App und ihr Abspielen innerhalb einer Playerumgebung sind durch den Unity-Programmcode definiert und werden im Absatz "Medienfunktionen" der Code-Dokumentation weiter beschrieben.

1.6 Das Redaktionssystem

Das Redaktionssystem (CMS) erleichtert die Einpflege von Inhalten.

Als CMS setzen wir **Drupal** ein, eine Software, die in PHP geschrieben ist, das Symfony-Framework nutzt und MySQL-Datenbanken generiert. Durch das Redaktionssystem wird die Einpflege des App-Contents und die Generierung der schon umfassend beschriebenen JSON-Dateien "tours.js" und "stations.js" vereinfacht.

Server-Anforderungen für das CMS

- PHP: Version 8.1 oder neuer
- MySQL: Version 8 oder neuer (alternativ eine aktuelle Version von MariaDB)
- RAM: 2GB
- Festplattenspeicher: 100 GB
- CPU: 2 Kerne

Wichtige Ergänzungen zum Hosting:

- Außerdem ist eine Administrationsoberfläche wie Plesk für die vereinfachte Einrichtung von Domains und Subdomains sowie Datenbanken, TLS-Zertifikate, etc. notwendig.
- Darüber hinaus benötigt der Webprogrammierer SSH-Zugang.
- Für das Deployment muss folgendes auf dem Server installiert und nutzbar sein:
 - Git (aktuelle Version)
 - Composer (Version 2)

Sollte einer dieser Voraussetzungen nicht gegeben sein, kann der Webprogrammierer nicht vernünftig veröffentlichen (deployen).

2. Code-Dokumentation zum Unity-Projekt

2.1 Frontend-Umsetzung

Für die Gestaltung und Programmierung von Benutzeroberflächen bietet Unity ein spezielles System namens **Unity UI** an. Dieses System ermöglicht es, Benutzeroberflächen zu erstellen, die in Form von 2D-Grafiken auf einem sogenannten **Canvas** dargestellt werden.

Arbeiten mit Canvas

Der Canvas ist das grundlegende Element für alle Benutzeroberfläche-Elemente in Unity und dient als eine Art digitale Leinwand, auf der andere Elemente platziert werden können.

Auf dem Canvas werden verschiedene **Benutzeroberfläche-Elemente** angeordnet. Dazu gehören unter anderem **Texte**, die zur Anzeige von Informationen oder Anleitungen genutzt werden, und **Bilder**, die entweder dekorative oder funktionale Zwecke erfüllen. Darüber hinaus können **Videos** über ein spezielles Video-Player-Element direkt auf dem Canvas abgespielt werden.

Ein Canvas kann flexibel an verschiedene Bildschirmgrößen angepasst werden, was besonders hilfreich ist, wenn eine Anwendung auf unterschiedlichen Geräten angezeigt werden soll. So entstehen **responsive Designs**. In diesen passen sich die Benutzeroberfläche-Elemente automatisch an unterschiedliche Bildschirmgrößen an und sehen auf jedem Gerät gut aus.

Die Anordnung der Benutzeroberfläche-Elemente erfolgt in einer Hierarchie, die es ermöglicht, mehrere Objekte zu gruppieren und sie als Einheit zu verwalten. Unity stellt verschiedene **Layout-Tools** zur Verfügung, wie das Grid Layout oder das Vertical Layout, die dabei helfen, die Elemente automatisch und konsistent auf dem Bildschirm anzuordnen.

Navigation durch Controller-Skripte

Um **Interaktivität** zu schaffen, können Canvas und seine Elemente durch Skripte ergänzt werden. Ein Beispiel hierfür wäre ein Bild, das bei einem Klick den Nutzer zu einer anderen Seite führt. Unity verwendet dafür die Programmiersprache C#.

Es wurden mehrere sogenannte "Sites" aufgebaut, welche die unterschiedlichen Seiten der App repräsentieren. Diese Seiten haben einen Canvas und darunter sind die einzelnen UI-Elemente wie Texte und Bilder platziert. Beispielsweise gibt es eine Startseite, wo man die Tour auswählt, eine Detailseite der Tour oder eine Seite für die Karte. Es wurde ein "**SiteController**" Script geschrieben, mit dessen Funktionen man von einer Seite zur anderen wechseln kann. Es gibt weitere Hilfsskripte wie "**TourController**" oder "**StationController**", die dafür verantwortlich sind, die Inhalte aus dem Drupal-Backend zu laden und im User Interface auf den Seiten anzuzeigen. Das **MapController**-Script ist für das Laden der Karte zuständig und ermöglicht die Auswahl und das Aufrufen der Stationen. Das **GalleryController**-Script lädt die Bildergalerie und das **VideoController**-Script ist für das Abspielen von Videos zuständig.

2.2 Standardfunktionen

Im Wesentlichen bietet die Unity-Entwicklungsumgebung nützliche Standards, die in unserer App zum Einsatz kommen. Dazu gehört vor allem das **ARFoundation-Framework**, welches

wir für unsere AR-Ausgabe nutzen. Aber auch die **Unity UI**, die wir für die Darstellung des User Interfaces nutzen. Desweiteren nutzen wir sogenannte externe Packages und Plugins, die uns Grundfunktionen zur Verfügung stellen. Dazu gehört das **Mapbox Plugin**, mit dessen Hilfe wir unsere Karte anzeigen können. Mithilfe des **AVProMovieCapture Plugins** können wir Videoaufnahmen machen. Mit **TextMeshPro** können wir Texte ausgeben und individuelle Styles hinzufügen.

2.3 Medienfunktionen

Medien (Texte, Bilder, Videos, 3D) sind zum Teil fest in der App hinterlegt und werden in der Benutzeroberfläche angezeigt. Zum anderen können sie flexibel aus dem Drupal-Backend geladen werden. Beim Start der App lädt die App dafür **JSON-Dateien** herunter, welche den Inhalt aus dem Backend referenzieren. Die JSON-Dateien enthalten Verlinkungen auf Bilder und Videos. Der Nutzer kann entscheiden, ob er diese Bilder und Videos beim Start direkt herunterladen möchte. Sie werden dann in den jeweiligen Stationen der App abgerufen und angezeigt. Zuständig für das Herunterladen der Inhalte ist das **DownloadContentController**-Script.

Es wurden benutzerdefinierte Controller-Skripte geschrieben, die alle eine spezifische Aufgabe zum Laden von Medien haben. Das **GalleryController**-Script lädt die Bildergalerie und das **VideoController**-Script ist für das Abspielen von Videos zuständig. Der **PoiInfoController** ist verantwortlich für das Laden der Informationsseite.

Desweiteren beinhaltet die App 3D-Dateien, die speziell aufbereitet und optimiert wurden, damit sie auf einem Smartphone wiedergegeben werden können. Das AR-Objekt der virtuellen Glashütte wird mithilfe des **GlasObjectController**-Scripts geladen und dargestellt.

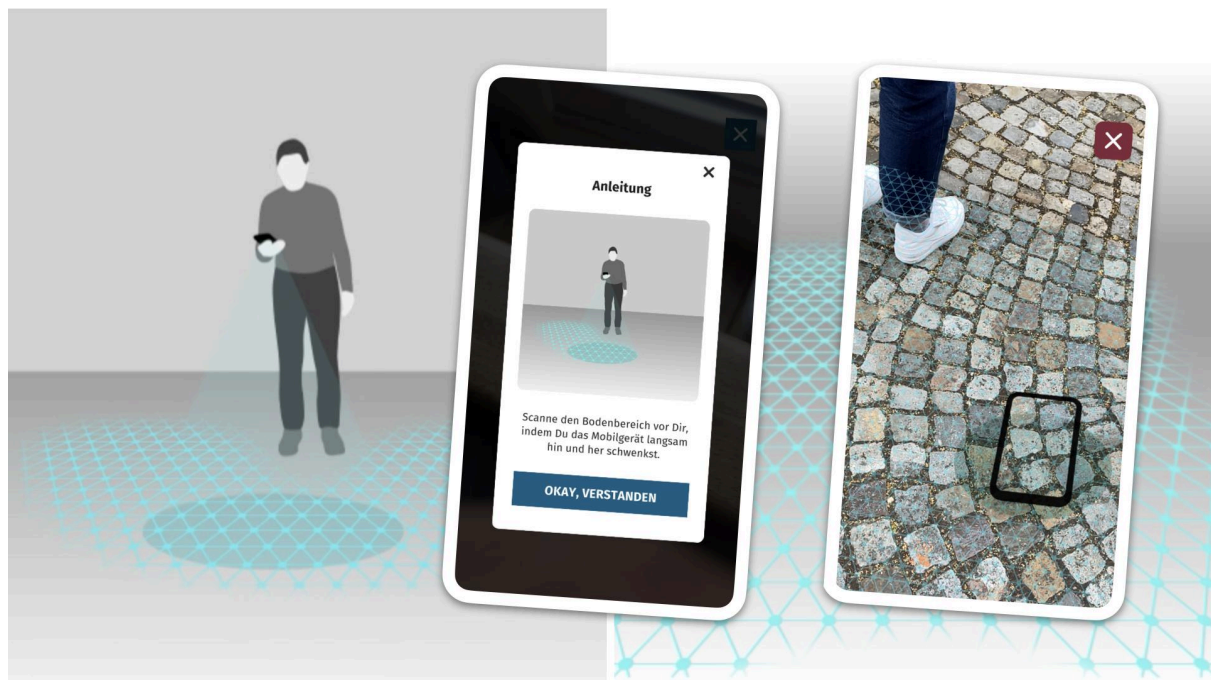
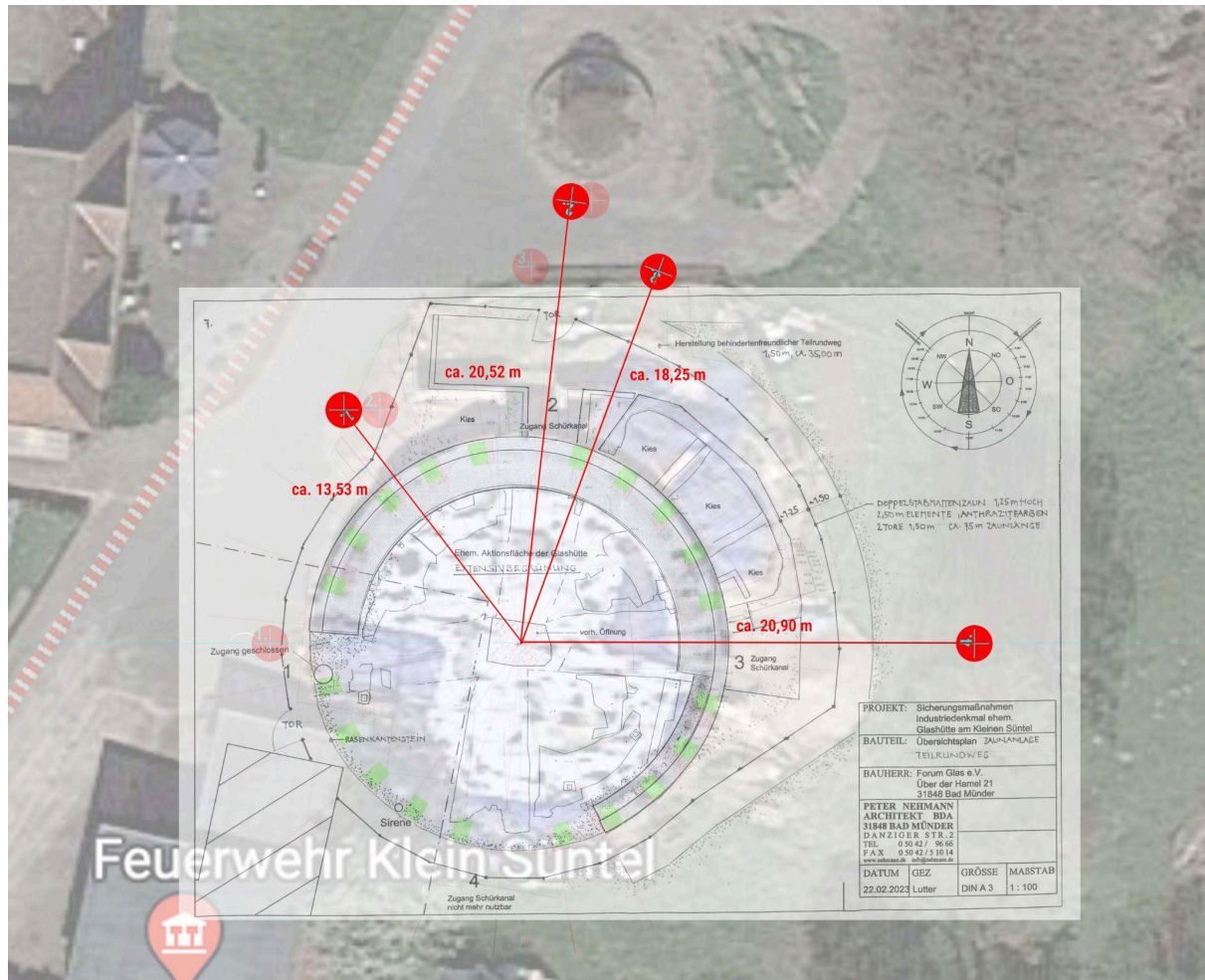
2.4 AR-Funktionen

Wir erklären im ersten Abschnitt des folgenden Textes zunächst das grundsätzliche Setting für die AR-Funktionen. Im zweiten Abschnitt werden wir fachlicher, nennen die Skripte und ihre Funktionen.

Grundlagen der AR-Ausspielung

In der App kombinieren wir für die Vor-Ort-Ausspielung des Rauchgaskegels zwei Trackingverfahren: Markertracking und Bodentracking. Durch die Kombination dieser Verfahren erreichen wir eine bestmögliche Präzision bei der Positionierung.

Markertracking: Für die Vor-Ort-Ausspielung des 3D-Modells nutzen wir vier im Gelände auf dem Boden platzierte QR-Codes als auslösende Marker. Unser 3D-Modell ist in fest hinterlegten Abständen an die Marker geheftet und zwar per x-, y- und z-Koordinaten in Relation zum Marker-Mittelpunkt.



Bodentracking: Während des Markertrackings wird aber parallel und unterstützend auch noch ein markerunabhängiges Verfahren eingesetzt, das Bodentracking. In diesem Verfahren wird im Kameramodus die Boden- b.z.w. Raumfläche vor den Nutzenden der App gescannt und dann von der App als Ausspielungsort für die Medieninhalte genutzt. Zur Ausspielung der ortsunabhängigen Variante des Rauchgaskegels und für den Aufruf des Avatars nutzen wir ausschließlich das Bodentracking.

Fachliche Erläuterungen

Unity stellt mit dem **ARFoundation-Framework** ein Tool zur Verfügung, mit dem sich AR-Inhalte wiedergeben lassen. Kernelement dafür ist eine **ARCamera**, die auf die Kamera des Smartphones zugreift und so zum einen das Kamerabild als Hintergrund ausgeben kann. Zum anderen kann sie virtuelle Elemente wie die Glashütte auf dem Kamerabild überlagern und so den Eindruck erwecken, als würde das 3D-Objekt in der Realität vor dem Nutzer stehen.

Die grundlegenden Funktionen für diese AR-Visualisierung werden vom ARFoundation-Framework zur Verfügung gestellt. Eigene Hilfscripte wie **ScanController** und **ARController** nutzen die Tools des ARFoundation-Frameworks, um für uns die gewünschte Darstellung zu ermöglichen. Die Kameradaten werden dabei kontinuierlich analysiert, um natürliche Umgebungsmerkmale wie Texturen oder Tiefendaten zu erkennen und zu nutzen. Diese Methode ist besonders nützlich für Anwendungen, die in dynamischen oder sich verändernden Umgebungen eingesetzt werden sollen. Sie bietet Flexibilität und ist ideal für ortsunabhängige AR-Erlebnisse, da sie dynamisch auf Veränderungen in der Umgebung reagieren kann. Mithilfe des **ScanController**-Scripts erhalten wir Positionsdaten für Bodenoberflächen, was uns wiederum ermöglicht, unseren virtuellen Glaskegel anhand dieser Positionsdaten auszurichten und vor dem User anzuzeigen. Die virtuellen Bodenoberflächen werden zusätzlich mithilfe des Scripts als Gitternetz visualisiert.

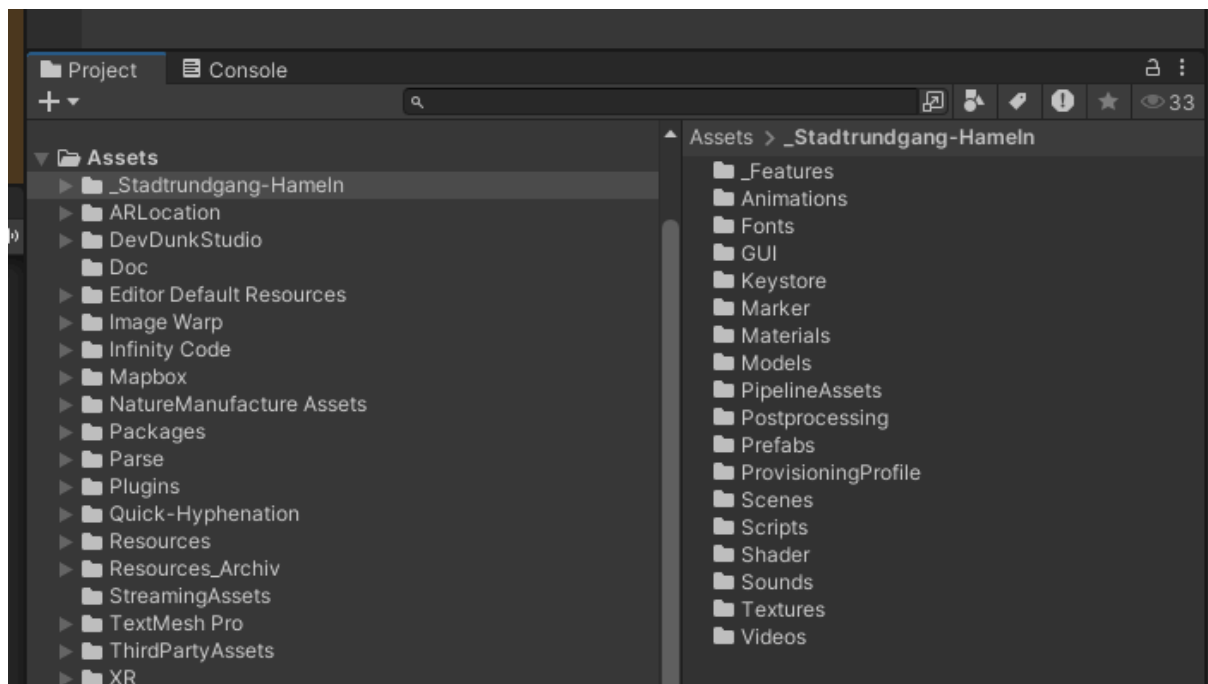
Das **GlasObjectController**-Script ist die zentrale Stelle für die Interaktionsmöglichkeiten mit der virtuellen Glashütte. Es ist verantwortlich für die korrekte Positionierung im Marker-Modus und bietet ebenfalls Funktionen zum Verschieben und Skalieren im Markerlosem-Modus. Zunächst erhalten wir dabei Positionsinformationen vom ARFoundation-Framework, wenn wir einen Marker scannen. Das **GlasObjectController**-Script nutzt diese Daten, um den virtuellen Glaskegel in einem bestimmten Abstand relativ zum Marker zu positionieren und anzuzeigen.

3. Datenstruktur-Dokumentation

3.1 Datenstruktur der Unity-App

Projekt- b.z.w. Verzeichnisstruktur

Sämtliche Projektdaten wie Scripte, Plugins und 3D-Daten werden in einen zentralen Assets-Ordner kopiert. Innerhalb des Assets-Ordners haben wir Unterordner, um unsere Daten geordnet abzulegen. Externe Plugins liegen dabei meistens direkt in einem eigenen Ordner unter unserem Assets-Ordner. Dazu gehört z. B. das **TextMesh Pro** Plugin, mit dem sich Text auf dem Canvas einfügen lässt. Ebenso haben wir das **Mapbox** Plugin, um eine Karte zu laden. Unsere eigenen Entwicklungen haben wir zentral in den "_Stadtrundgang-Hameln"-Ordner abgelegt. Wir haben z. B. einen **GUI-Ordner**, wo wir alle User Interface Bilder speichern. Ebenso haben wir einen **Scripts-Ordner**, wo wir den primären Quellcode der App halten.



3.2 Datenstruktur des CMS

Die eigentlichen Funktionen der App und ihr Zusammenspiel mit den JSON-Dateien sind **unabhängig von dem Redaktionssystem**, das dem Kern des Systems nur angliedert ist, siehe auch die entsprechenden Abbildungen im Absatz "1. Die Systemarchitektur".

Das Redaktionssystem wurde nicht von uns selbst (Expo-Etage I Mobile-etage GmbH) umgesetzt, sondern von der Digitalagentur Hob by Horse GmbH (www.hob-by-horse.de), auf die wir bei Nachfragen und für Anpassungen kostenpflichtig zugreifen müssen. Wir haben keinen Adminzugriff auf das System und können es nicht in der Tiefe beschreiben und übergeben, wie unseren funktionalen Leistungskern: Die Unity-Programmierung, siehe

auch den Abschnitt "Code-Dokumentation" und die JSON-Schnittstelle, siehe vorangegangenen Absatz "Die JSONs".

Gerne stellen wir trotzdem zum besseren Verständnis das ursprüngliche Datengliederungskonzept zur Verfügung, das 2021 Grundlage für die Entwicklung des Redaktionssystems war, siehe das PDF "Datengliederung-2021":

- <https://raw.githubusercontent.com/MobileEtage/Ha-Py-Virtuell/main/demo/datengliederungskonzept-2021.pdf>

Weil letztendlich alle contentbezogenen Erweiterungen der App über die JSONs erfolgen, steht für den Kunden die Entscheidung an, ob er für etwaige Weiterentwicklungen und Ausbau der App das aktuell eingebundene Redaktionssystem Drupal nutzen möchte, oder ob er sich andere Zugriffe auf die JSONs erschließen will, beispielsweise manuell oder über ein anderes Redaktionssystem.

4. Versionsverwaltung

4.1 Versionierung der Unity-App

Wir nutzen als Repository und zur Versionsverwaltung des Unity-Projekts das in Unity integrierte **“Plastic SCM”**. Plastic SCM ist ein Versionskontrollsystem für die Verwaltung von Softwareprojekten, wobei der Schwerpunkt auf den Bedürfnissen von Teams liegt, die an Spielen und 3D-Projekten arbeiten.

Unity Technologies hat Plastic SCM in seine Suite integriert, um den Entwicklungsprozess in Unity-basierten Projekten zu optimieren. Plastic SCM bietet fortschrittliche Funktionen für das Branching und Merging, die Entwicklern helfen, effizient mit mehreren Versionen ihrer Projekte zu arbeiten. Die nahtlose Integration von Plastic SCM in Unity ermöglicht es Entwicklern, direkt in der Unity-Umgebung auf Versionskontrollfunktionen zuzugreifen, was den Workflow erheblich verbessert.

Man kann anderen Unity-Entwicklern Zugriff auf seine Projekte geben und dabei Rechte einstellen. So ist es möglich, dass man nur Leserechte vergibt, um unbeabsichtigte Änderungen am Projekt zu unterbinden.

4.2 Versionierung des CMS

Die aktuell installierte Version des Redaktionssystems ist:

- Drupal core (Sicherheitsupdate) 10.0.2

Die Module sind in folgenden Versionen installiert:

- Admin Toolbar 3.3.0
- Chosen 3.0.5
- Entity Reference Revisions 8.x-1.10
- Field Group 8.x-3.4
- Gin Toolbar 8.x-1.0-rc1
- Gin Admin Theme 8.x-3.0-rc1

5. Übergabe der Projektergebnisse

Wir wollen der Zielsetzung des Projekts gerecht werden und eine Weiternutzung und Weiterentwicklung unserer Entwicklungen ermöglichen.

5.1 Übergabe des Unity-Projekts

- (1) Wir haben am 25.01.2025 von dem gesamten Unity-Projekt ein Zip-Paket erstellt und dieses an unseren Kunden übermittelt. Es ist vollumfänglich funktionstüchtig, erweiterbar und als App exportierbar. Die Bearbeitung etwaiger Nachfragen und das Geben von Hilfestellungen zu fachlichen Details (sprich: ein umfänglicher Support) ist aber weder Teil dieses Projekts noch der Übergabe und muss gesondert in Rechnung gestellt werden.
- (2) Wir haben das Unity-Projekt außerdem auf GitHub veröffentlicht:
 - <https://github.com/MobileEtage/Ha-Py-Virtuell/tree/main/HamelN-Pyrmont-Virtuell>

5.2 Übergabe der JSONs

Im Rahmen der Dokumentation übergeben wir die JSON-Dateien **“tours.json”** und **“stations.json”** in mehreren “Versionen”. Wie auch im Kapitel “1.4 Die JSON-Schnittstelle” erläutert, sind die JSONs im weitesten Sinne einfach zwei Textdateien, deren Dateiendung “json” lautet. Diese Dateien sind ihrer Natur nach Momentaufnahmen, da sie die aktuellen Medieninhalte der App teils in Klartext enthalten, teils durch IDs oder URLs referenzieren.

Die JSONs (Stand 07.01.2025) im Repository zur Ansicht:

- <https://github.com/MobileEtage/Ha-Py-Virtuell/blob/main/demo/tours.json>
- <https://github.com/MobileEtage/Ha-Py-Virtuell/blob/main/demo/stations.json>

Die JSONs (Stand 07.01.2025) im Repository zum Herunterladen:

- <https://raw.githubusercontent.com/MobileEtage/Ha-Py-Virtuell/main/demo/tours.json>
- <https://raw.githubusercontent.com/MobileEtage/Ha-Py-Virtuell/main/demo/stations.json>

Die kommentierten JSONs (Stand 07.01.2025) im Repository zum Herunterladen:

- <https://raw.githubusercontent.com/MobileEtage/Ha-Py-Virtuell/main/demo/tours-kommentiert.pdf>
- <https://raw.githubusercontent.com/MobileEtage/Ha-Py-Virtuell/main/demo/stations-kommentiert.pdf>

Aufruf der “Live-JSONs” über den Browser: Die zwei JSONs in ihrer aktuellen Live-Version können auch über einen Browser aufgerufen und betrachtet werden, allerdings aus Sicherheitsgründen nur, wenn die Nutzenden in der Rolle eines Admins oder Editors im Redaktionssystem eingeloggt sind.

- <https://hameln.stadtrundgang.dev.hob-by-horse.de/api/tours.json>
- <https://hameln.stadtrundgang.dev.hob-by-horse.de/api/stations.json>

5.3 Übergabe des Redaktionssystems (Schulung)

Zur Übergabe des Redaktionssystems hat am 31.01.2025 eine projektbezogene Online-Schulung stattgefunden. In dieser wurde den Teilnehmern gezeigt, wie man neue Touren, neue Stationen und neue Medieninhalte anlegt. Das System ist intuitiv und weitgehend selbsterklärend. Man sollte beachten, dass alle darin gemachten Änderungen unmittelbar, d.h. ohne Previews und etwaige Freigabeschleifen, in der App zur Verfügung stehen.

Wir verfügen nicht über eine ausführliche, schriftliche Dokumentation ("Handbuch") zum Redaktionssystem und können eine solche im Rahmen dieses Projekts aufgrund des Aufwandes auch nicht erstellen. Einige wenige Zusammenhänge und Besonderheiten möchten wir jedoch hier schriftlich fixieren.

Das Redaktionssystem Drupal:

- <https://hameln.stadtrundgang.dev.hob-by-horse.de/>
Zugangsdaten wurden separat übermittelt.

5.3.1 Fokus auf "Inhalte"

Drupal ist ein hochkomplexes Redaktionssystem, dass eigentlich für die Erstellung von Webseiten entwickelt wurde. In unserem Kontext wird es "nur" für die Erstellung strukturierter Inhalte eingesetzt. Deshalb liegen viele potentielle (aber kontextferne) Funktionalitäten brach. Klicken Sie auf der ersten Seite links oben auf "Inhalte".

Inhalte (Übersichtsseite)

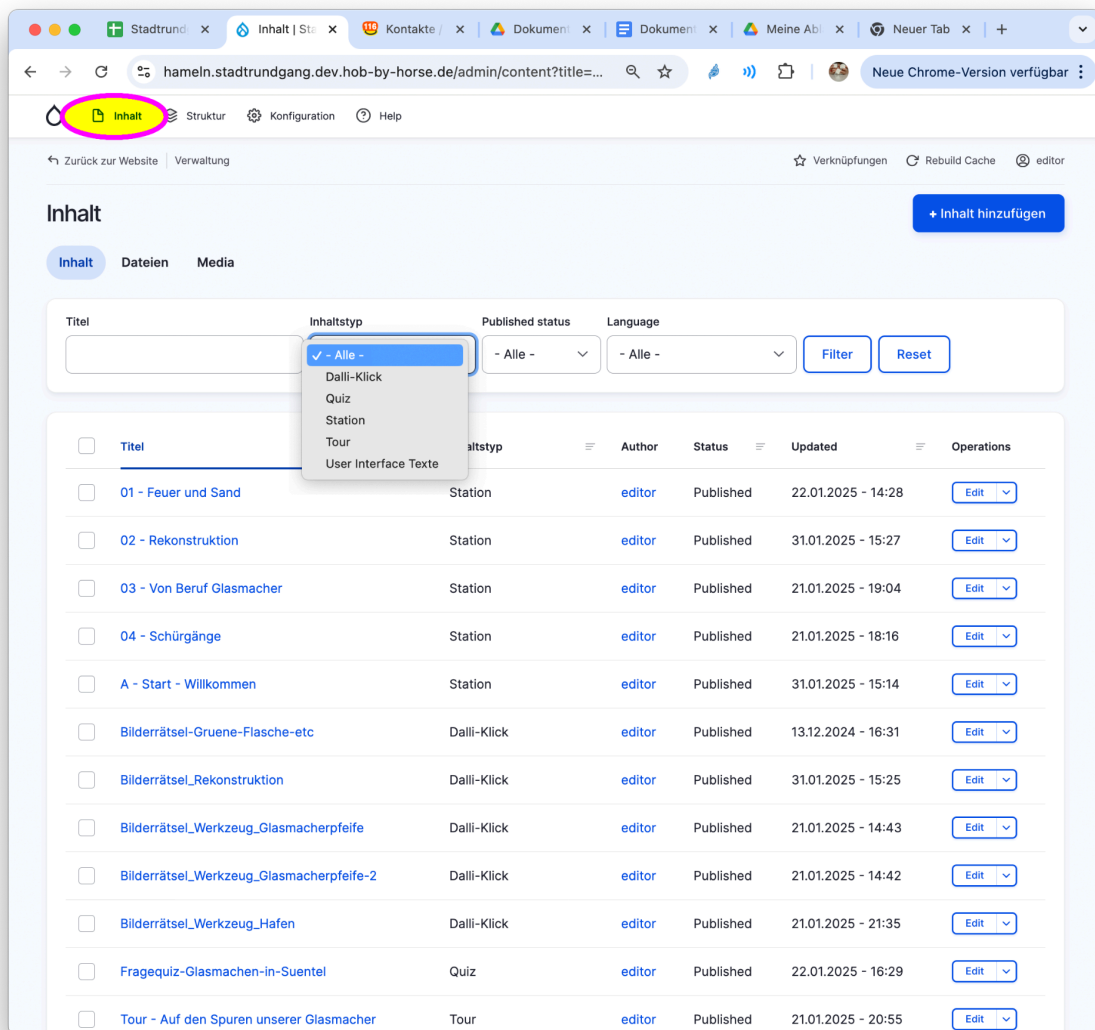
"Inhalte" ist der einzige Bereich des Redaktionssystems, der für die Erstellung der App-Inhalte relevant ist. Hier werden sämtliche Inhalte aufgelistet, die für die App schon erstellt wurden.

Sie können geöffnet und über "edit" bearbeitet werden.

Über "+ Inhalt hinzufügen" können neue Inhalte ergänzt werden.

Folgende Inhaltstypen sind hinterlegt:

- Tour
- Station
- Dalli-Klick (Bilderrätsel)
- Quiz (Fragerätsel)
- User Interface Texte



5.3.2 Neue Touren anlegen

Die Basiseingaben zur Tour

Im oberen Bereich des Touren-Formulars werden Angaben abgefragt, die sich im Ergebnis, also in der App, auf folgende zwei Seiten verteilen:

- **Touren-Übersichtsseiten:** Hier werden die Touren der App gezeigt werden
- **Touren-Detailseite:** Hier werden die Stationen der Tour gelistet

Besonderheit: Die Bildhöhe der Touren-Teaserbildes bestimmt die Gesamthöhe der Touren-Kachel. Wenn also mehrere Touren hinterlegt werden, kann durch Anpassungen an den Touren-Bildern die Erscheinung der Tourenseite maßgeblich mitbestimmt werden. Auch "optische Priorisierungen" können darüber gesteuert werden.

Stationen-Verknüpfung zur Tour

Im unteren Bereich des Formulars werden die Stationen mit der Tour verknüpft. Es stehen ausschließlich schon vorher angelegte Stationen zur Auswahl b.z.w. zur Verfügung. Diese werden über ihren ("internen") Titel aufgerufen.

5.3.3 Neue Stationen anlegen b.z.w. verknüpfen

Die Basiseingaben zur Station

Im oberen Bereich des Stationen-Formulars werden Basisangaben abgefragt.

Diese Basiseingaben verteilen sich in der App auf drei Seiten:

- **Listenelement zur Station:**
Die Station wird innerhalb der Touren-Detailseite als Listenelement angezeigt.
- **Karten-Bubble zur Station:**
Die Station wird innerhalb der Karte als Bubble platziert.
- **Overlay-Seite zur Station:**
Die Station wird mithilfe eines "Standortbildes" auf einer Overlay-Seite angekündigt.
Die App-Nutzenden erhalten auf dieser Seite außerdem die Optionen "QR-Code scannen" und "Weiter ohne QR-Code".

5.3.4 Neue Medieninhalte anlegen b.z.w. verknüpfen

Neuen Medieninhalte werden innerhalb des Inhaltstyps Station eingepflegt oder verknüpft.

Dies geschieht über drei mögliche Wege:

- **(1) Programmiertechnische Verknüpfung über die ID der Station.**
Auf diesem Weg sind die aktuellen Startinhalte an die Stationen angebunden.
Im Falle der Glasmacher-Tour sind das der Avatar und das 3D-Modell des Rauchgaskegels. Deshalb dürfen die IDs der schon eingerichteten Stationen auch nicht verändert werden.
- **(2) Direkteinpfege von Medieninhalten** in das große Stationen-Formular.
Folgende Medien werden im Ergebnis im **3-Punkte-Menü** der App angezeigt:
 - **Information (Info-Icon)**
 - **Bildergalerie (Image-Icon)**
 - **Video (Video-Play-Icon)**
 - **Audio-Beitrag (Audio-Icon)**
 - **360-Grad-Ansichten (360-Grad-Icon)**
 - **AR-Icons (AR-Icon)**
- **(3) Verknüpfung von Spielen;** Diese werden nicht innerhalb der Stationen eingepflegt, sondern als eigenständige Inhaltstypen angelegt (siehe nächster Abschnitt) und in den Stationen nur verknüpft. Ausgabe im 3-Punkte-Menü:
 - **Spiel (Pokal-Icon)**

5.3.4 Neue Spiele anlegen und mit den Stationen verknüpfen

Aus programmiertechnischen Gründen werden Spiele **nicht innerhalb** der Stationen angelegt, sondern als **eigener Inhaltstyp**. Das bringt unter anderem den Vorteil mit sich, dass Spiele auch an mehreren Stationen ausgespielt werden könnten.

Hinterlegt sind die Typen Bilderrätsel ("Dalli-Klick") und Fragerätsel ("Quiz").

- **Besonderheiten des Inhaltstyps "Dalli-Klick" (Bilderrätsel)**
 - Jedes zu erratende Bild wird als einzelnes Bilderrätsel angelegt, das typischerweise innerhalb der Stationen mit weiteren (inhaltlich verwandten) Bilderrätsel-Einheiten "in Reihe geschaltet" wird. Die Programmierung erkennt

diese Reihen und spielt den Abschlußscreen aller Spiel erst am Ende der Reihe aus.

- Die Spielelogik wird in einem
- **Besonderheiten des Inhaltstyps “Quiz” (Fragerätsel)**
 - Die Fragen plus Antwortoptionen werden innerhalb dieses Inhaltstyps eingepflegt.
- **Gemeinsame Spiellogik beider Games:**
 - Man kann **pro Frage zwischen 2 und 4 Kurzantworten** hinterlegen. Für diesen Inhaltstyp empfehlen wir aufgrund des sehr begrenzten Platzes im Idealfall nur ein einziges kurzes Wort pro Option. Unabhängig davon: Die Programmierung stellt die Schriftgröße so ein, dass der Text in die Kacheln passt. Das kann aber sehr schnell eine deutlich zu kleine Schrift ergeben, sodass dann redaktionell mit Textkürzung nachgesteuert werden muss. Trotzdem ggf. hilfreich: Die 2 bis 4 Kurzantworten können in der Eingabemaske des Formulars durch Einfügen des HTML-Tags “
” typografisch minimal gestaltet werden. Beispiele:

Antwort DE *

Verfügbare Roh- & Brenn-
stoffe, gute Nachfrage

Antwort DE *

Zentrale Lage, gute Verkehrs-
anbindung

- Es gibt zu jeder Frage **nur eine richtige Antwort** (alle anderen Antwortoptionen sind falsch). Die richtige Antwort muß im CMS immer an erster Stelle (!) eingepflegt werden. Dadurch ist die Antwortoption für die Programmierung als korrekte Antwort erkennbar. Für die Ausgabe in der App werden natürlich alle Antwortoptionen pro Aufruf des Rätsels an unterschiedlichen Positionen platziert.
- Auf einer Feedbackseite erfahren die Spielenden, ob ihre Antwort-Auswahl korrekt oder nicht korrekt war. Außerdem wird eine inhaltliche Erläuterung **zur richtigen Wahl** ergänzt (in Drupal heißt das Feld zur Hinterlegung dieser Erläuterung “Beschreibung”). Dadurch erfahren auch die Spielenden, die nicht korrekt geantwortet haben, die inhaltlichen Hintergründe der korrekten Antwort.

5.3.5 Vorgaben für die Medieninhalte

Die Medieninhalte (Bilder, Audios und Videos) müssen vor der Integration fachgerecht vorbereitet sein, was fundierte Kenntnisse und die Beherrschung diverser Programme erfordert. Wir haben die entsprechenden Vorgaben aufgrund ihrer Bedeutung in den eigenen Abschnitt “Vorgaben für Medieninhalte” ausgelagert.

Von der Einhaltung der Vorgaben für die Medieninhalte hängt nicht nur das Aussehen, sondern indirekt auch die Funktion der App ab, da keine Instanzen zur Überprüfung der Vorgaben in das Redaktionssystem integriert sind. So könnten beispielsweise unkomprimierte, riesige Dateien die App zum Absturz bringen.

Alle Zahlenangaben in Pixel.

Bild für Inhaltstyp: Tour Höhe des Bildes bestimmt Höhe der Tourkachel Bild der Tour: Maximal: 2048 x 2048 Empfohlen: 1080 x 900 Dateigröße: < 200 kb	Bilder für Inhaltstyp: Station Standortbild: Maximal: 1024 x 1024 Empfohlen: 1024 x 768 Dateigröße: < 100 kb Listenbild: Maximal: 256 x 256 Empfohlen: 256 x 256 Dateigröße: < 50 kb
Feature "Guide" Video Hintergrundbild Maximal: 2048 x 2048 Empfohlen: 1080 x 1920 Dateigröße: < 200 kb Audio Hintergrundbild Maximal: 2048 x 2048 Empfohlen: 1080 x 1920 Dateigröße: < 200 kb	Feature "Info" Hauptbild Maximal: 2048 x 2048 Empfohlen: 1024 x 768 Dateigröße: < 200 kb Bildliste Maximal: 2048 x 2048 Empfohlen: 1024 x 768 Dateigröße: < 100 kb
Feature "Audio" Bild Startseite Maximal: 1024 x 1024 Empfohlen: 512 x 512 Dateigröße: < 50 kb	Feature "Spiel" Bild Startseite Maximal: 1024 x 1024 Empfohlen: 512 x 512 Dateigröße: < 50 kb
Feature "AR" Bild Startseite Maximal: 1024 x 1024 Empfohlen: 512 x 512 Dateigröße: < 50 kb Video Maximal und empfohlen: 1920 x 1080 Dateigröße: < 20 MB Bitte gutes Komprimierungsprogramm verwenden, z. B. das kostenlose "Handbrake".	Bilder für Inhaltstyp: Dalli-Klick Zu erratendes Bild Maximal: 2048 x 2048 Empfohlen: 1024 x 1024 Dateigröße: < 100 kb Ergebnisbild: Maximal: 2048 x 2048 Empfohlen: 1024 x 1024 Dateigröße: < 100 kb

Feature "Panorama"

360° Panorama Bild

Maximal: 4096 x 4096

Empfohlen: 4096 x 2048

Dateigröße: < 500 kb

Bild Startseite

Maximal: 1024 x 1024

Empfohlen: 512 x 512

Dateigröße: < 50 kb

Falls Detailfoto

(Sonderanwendung!)

Empfohlen: 1024 x 1091

Minimal: 460 x 490

Dateigröße: < 100 kb